

Capstone Project SMS Spam Detection

Prepared by: **Ali Lakzaei**, Teaching Assistant

Course: Applied Machine Learning

Instructor: Hossein Homaei

Project Overview

This project focuses on building a machine learning system to classify SMS messages as either **spam** or **ham** (legitimate messages). Students will work through the complete machine learning pipeline, from data preprocessing to model deployment and evaluation.

Dataset: SMS Spam Collection Dataset

Objective: Develop and optimize a classification model to accurately distinguish between spam and legitimate SMS messages.

Learning Objectives

Upon completion of this project, students will be able to:

- Apply data cleaning and normalization techniques to text data
- Perform exploratory data analysis (EDA) and create meaningful visualizations
- Select appropriate machine learning models for text classification
- Optimize model hyperparameters and improve performance
- Evaluate models using appropriate metrics and interpret results
- Present technical work effectively to peers and instructors

Project Structure

The project is divided into **5 main phases**, each building upon the previous work:

Phase 01: Data Cleaning & Normalization

Objectives:

- Load and inspect the raw dataset
- Identify and handle missing values, duplicates, and inconsistencies
- Normalize text data (lowercasing, removing special characters, etc.)
- Perform tokenization[†] and text preprocessing
- Handle class imbalance if present
- Split data into training, validation, and test sets

†Note on Tokenization: Tokenization is the process of breaking down text into smaller units called tokens, which are typically words or subwords. For example, the sentence "Hello world!" would be tokenized into ["Hello", "world"]. This is a fundamental step in text preprocessing that allows machine learning models to process and analyze text data by converting raw text into a structured format that algorithms can understand. Tokenization helps in standardizing text input and is essential for feature extraction methods like Bag of Words and TF-IDF.

Deliverables:

- Cleaned dataset with documentation of all transformations
- Preprocessing pipeline code
- Summary report of data quality issues found and solutions applied

Key Tasks:

- ☐ Load the CSV file and examine its structure
- ☐ Check for missing values, duplicates, and data types
- ☐ Analyze label distribution (spam vs. ham)
- ☐ Implement text normalization (lowercase, remove punctuation, handle special characters)
- ☐ Remove or handle empty messages
- ☐ Create train/validation/test splits
- ☐ Document all preprocessing steps

Evaluation Criteria:

- Proper handling of data quality issues
- Appropriate text normalization techniques
- Correct data splitting methodology
- Clear documentation of preprocessing steps

Phase 02: Data Understanding & Visualization

Objectives:

- Perform exploratory data analysis (EDA)
- Visualize class distribution and message characteristics
- Analyze text features (length, word count, character frequency, etc.)
- Identify patterns and insights that inform model selection
- Create informative visualizations

Deliverables:

- EDA report with visualizations
- Statistical summary of the dataset
- Insights and observations document

Key Tasks:

- ☐ Analyze class distribution (spam vs. ham ratio)
- ☐ Visualize message length distribution (characters, words) for each class
- ☐ Analyze most common words in spam vs. ham messages
- ☐ Create word clouds for spam and ham messages
- ☐ Analyze frequency of special characters, numbers, URLs, etc.
- ☐ Identify distinguishing features between spam and ham messages:
 - ☐ Compare statistical differences (mean, median, variance) in message length, word count, and character frequencies between classes
 - ☐ Analyze presence and frequency of specific patterns (e.g., phone numbers, currency symbols, urgency words like "FREE", "WIN", "URGENT")
 - ☐ Use statistical tests (e.g., t-tests, chi-square tests) to identify features with significant differences between spam and ham
 - ☐ Create comparative visualizations (side-by-side comparisons, overlays) to highlight differences
- ☐ Create correlation analysis if applicable
- ☐ Document key insights and patterns

Visualizations to Include:

- Bar chart: Class distribution
- Histogram: Message length distribution (by class)
- Box plot: Message length comparison
- Word frequency charts (top N words per class)
- Word clouds (spam vs. ham)
- Analysis of special characters/numbers/URLs frequency

Evaluation Criteria:

- Comprehensive EDA covering multiple aspects
- Clear and informative visualizations
- Meaningful insights derived from the analysis
- Professional presentation of findings

Phase 03: Model Selection

Objectives:

- Implement multiple machine learning models
- Compare baseline models using appropriate metrics
- Select promising models for further optimization
- Justify model choices based on data characteristics

Deliverables:

- Implementation of at least 3-4 different models
- Model comparison report
- Selected models with justification

Key Tasks:

- ☐ Choose appropriate text feature extraction methods:
 - ☐ Bag of Words (Count Vectorizer)
 - ☐ TF-IDF (Term Frequency-Inverse Document Frequency)
 - ☐ Word embeddings (optional: Word2Vec, GloVe, or pre-trained embeddings)
- ☐ Implement baseline models:
 - ☐ Naive Bayes (MultinomialNB or BernoulliNB)
 - ☐ Logistic Regression
 - ☐ Support Vector Machine (SVM)
 - ☐ Random Forest (optional)
 - ☐ Neural Network (optional, for advanced students)
- ☐ Train each model on the training set
- ☐ Evaluate on validation set using multiple metrics:
 - ☐ Accuracy
 - ☐ Precision, Recall, F1-Score
 - ☐ Confusion Matrix
 - ☐ ROC-AUC (if applicable)
- ☐ Compare models and select top 2-3 for optimization
- ☐ Document model selection rationale

Models to Implement (Minimum):

1. **Naive Bayes** (MultinomialNB)
2. **Logistic Regression**
3. **Support Vector Machine (SVM)**

Optional Models (for bonus points):

- Random Forest
- Neural Network (MLP or simple deep learning)
- Ensemble methods

Evaluation Criteria:

- Correct implementation of feature extraction
- Proper model training and validation
- Appropriate metric selection and interpretation
- Clear comparison and justification of model choices

Phase 04: Model Refinement & Optimization

Objectives:

- Optimize hyperparameters for selected models
- Implement feature engineering improvements
- Address class imbalance if needed
- Improve model performance through iterative refinement

Deliverables:

- Optimized models with tuned hyperparameters
- Comparison of before/after optimization results
- Documentation of optimization process

Key Tasks:

- ☐ Perform hyperparameter tuning:
 - ☐ Grid Search or Random Search
 - ☐ Cross-validation (k-fold, e.g., 5-fold)
 - ☐ Optimize for appropriate metric (F1-score recommended for imbalanced data)
- ☐ Experiment with different feature extraction parameters:
 - ☐ n-gram ranges (unigrams, bigrams, trigrams)
 - ☐ Maximum features
 - ☐ Min/max document frequency
- ☐ Address class imbalance (if present):
 - ☐ SMOTE or other oversampling techniques
 - ☐ Class weights in models
 - ☐ Undersampling (if appropriate)
- ☐ Feature engineering:
 - ☐ Add message length as a feature
 - ☐ Add count of special characters, numbers, URLs
 - ☐ Experiment with feature combinations
- ☐ Iterate and refine based on validation results
- ☐ Document all optimization steps and their impact

Optimization Techniques:

- Hyperparameter tuning (GridSearchCV/RandomSearchCV)
- Cross-validation
- Feature selection/engineering
- Class imbalance handling
- Ensemble methods (optional)

Evaluation Criteria:

- Systematic approach to hyperparameter tuning
- Improvement in model performance
- Proper use of cross-validation
- Clear documentation of optimization process
- Justification of final model choices

Phase 05: Model Inference & Model Evaluation

Objectives:

- Evaluate final models on the test set
- Perform comprehensive model evaluation
- Analyze model errors and limitations
- Create a simple inference pipeline
- Prepare final presentation

Deliverables:

- Final model evaluation report
- Test set results with detailed metrics
- Error analysis
- Inference pipeline/demo
- Final presentation

Key Tasks:

- ☐ Evaluate final optimized models on test set:
 - ☐ Calculate all relevant metrics
 - ☐ Generate confusion matrices
 - ☐ Create ROC curves (if applicable)
 - ☐ Calculate precision-recall curves
- ☐ Perform error analysis:
 - ☐ Identify common misclassification patterns
 - ☐ Analyze false positives and false negatives
 - ☐ Understand model limitations
- ☐ Create inference pipeline:
 - ☐ Function to preprocess new messages
 - ☐ Function to make predictions
- ☐ Test inference on sample messages
- ☐ Prepare final presentation covering:
 - ☐ Project overview and objectives
 - ☐ Data exploration findings
 - ☐ Model selection and optimization process
 - ☐ Final results and evaluation
 - ☐ Conclusions and future work

Evaluation Metrics (Final):

- Accuracy
- Precision, Recall, F1-Score (macro and weighted)
- Confusion Matrix
- ROC-AUC Score
- Precision-Recall AUC
- Classification Report

Evaluation Criteria:

- Comprehensive test set evaluation

- Deep error analysis and insights
- Working inference pipeline
- Professional presentation
- Clear communication of results and methodology

Dataset Information

Dataset Source: Download the [spam-sms-classification](#) dataset from Kaggle. A copy of this dataset is also available at the Github repository of the course.

Structure:

- **v1:** Label column (ham/spam)
- **v2:** SMS message text
- Additional empty columns (can be ignored)

Dataset Characteristics:

- Binary classification problem
- Text data requiring NLP preprocessing
- Potential class imbalance (typical in spam detection)

Technical Requirements

Programming Language & Tools:

- Python 3.x (recommended: 3.8+)
- Jupyter Notebook or Python scripts for development
- Git for version control (recommended)

Required Libraries:

- Data Processing: pandas, numpy
- Text Processing: nltk, scikit-learn, re
- Machine Learning: scikit-learn
- Visualization: matplotlib, seaborn, wordcloud
- Optional: tensorflow/keras, spacy (for advanced models)

Code Organization:

Organize your project with clear structure (Example):

```
project/
├── data/
│   ├── raw/
│   │   └── spam.csv
│   └── processed/
└── notebooks/
```

```
├── 01_data_cleaning.ipynb
├── 02_eda_visualization.ipynb
├── 03_model_selection.ipynb
├── 04_model_optimization.ipynb
├── 05_evaluation_inference.ipynb
├── src/
│   ├── preprocessing.py
│   ├── models.py
│   └── inference.py
├── results/
│   ├── visualizations/
│   └── model_artifacts/
└── README.md
```

Evaluation

Overall Project Assessment (100 points)

Phase	Points	Criteria
Phase 01: Data Cleaning	15	Quality of preprocessing, handling of issues, documentation
Phase 02: EDA & Visualization	20	Comprehensiveness, quality of visualizations, insights
Phase 03: Model Selection	20	Correct implementation, comparison, justification
Phase 04: Optimization	20	Systematic approach, performance improvement, documentation
Phase 05: Evaluation & Presentation	25	Comprehensive evaluation, error analysis, presentation quality

Bonus Points (up to 10):

- Advanced models (neural networks, ensembles)
- Creative feature engineering
- Deployment/demo application
- Exceptional visualizations or insights

Timeline & Milestones

Recommended Timeline (5 weeks)

Week	Phase	Deliverable
1	Phase 01	Cleaned dataset & preprocessing report
2	Phase 02	EDA report with visualizations
3	Phase 03	Model comparison report
4	Phase 04	Optimized models & optimization report

Week	Phase	Deliverable
5	Phase 05	Final evaluation & presentation preparation

Submission Requirements

Code Submission:

- All Jupyter notebooks or Python scripts
- Well-commented and organized code
- Requirements.txt file with all dependencies
- README.md with setup instructions

Documentation:

- Report for each phase (can be in notebooks or separate documents)
- Final comprehensive report covering all phases
- Presentation slides (15-20 minutes)

Final Deliverables:

- Code
- Final report (PDF) covering all 5 phases
- Presentation (PowerPoint/PDF) for final presentation
- Working inference pipeline (demonstrable)

Best Practices & Tips

Code Quality:

- Write clean, readable, and well-commented code
- Use functions and classes for reusable components
- Follow PEP 8 style guidelines
- Use version control (Git)

Documentation:

- Document all assumptions and decisions
- Explain why you chose specific approaches
- Include references to papers/methods used
- Keep notebooks organized with clear sections

Experimentation:

- Keep a log of experiments and their results
- Save model artifacts and results
- Use random seeds for reproducibility
- Document hyperparameters and configurations

Presentation:

- Tell a story: problem → approach → results → insights
- Use clear visualizations
- Explain technical concepts
- Practice your presentation timing

Resources & References

Python Programming Language:

- Documentation: <https://www.python.org/doc/>
- Style Guide: <https://peps.python.org/pep-0008/>

Text Preprocessing:

- NLTK Documentation: <https://www.nltk.org/>
- Scikit-learn Text Feature Extraction: https://scikit-learn.org/stable/modules/feature_extraction.html

Machine Learning:

- Scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html
- A. Geron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, 3rd ed. O'Reilly Media, 2023.

Questions & Support

For questions or clarifications, please contact the course teaching assistant during office hours or via course communication channels.

Academic Integrity

- All work must be your own
- Cite all sources and references
- Collaboration on understanding concepts is encouraged, but code and reports must be individual work
- Use of AI tools (ChatGPT, etc.) must be disclosed and properly cited

Good luck with your project!

This project is designed to give you hands-on experience with the complete machine learning pipeline. Take your time, experiment, and most importantly, learn from the process.